CI/CD Pipeline for Python Energy Management Application using Docker

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no.	Student Name	Enrollment No.
1.	Naina Manghani	EN22CS301626
2.	Mahak Shrimal	EN22CS301566
3.	Mahak Taose	EN22CS301567
4.	Mohit Patel	EN22CS301610
5.	Paavan Patni	EN22CS301671

Group Name:Group 02D6 Project

Number:DO-13

Industry Mentor Name:

University Mentor Name:Prof.Akshay Saxena

Academic Year:2025-2026

TABLE OF CONTENTS

S.NO.	TOPICS	PAGE NO.
	TITLE PAGE	1
	TABLE OF CONTENTS	2-3
	LIST OF FIGURES	4
1.	**Chapter 1:PROBLEM STATEMENT AND OBJECTIVES**	5
	1.1 Problem Statement	5-6
	1.2 Project Objectives	6-7
	1.3 Scope of the Project	7-8
2.	**Chapter 2:PROPOSED SOLUTION**	9
	2.1 Key Features	9
	2.2 Overall Architecture / Workflow	10
	2.2.1 System Architecture	10-12
	2.2.2 DFD	12-14
	2.2.3 Use Case	14-15
	2.2.4 Class Diagram	15-16
	2.2.5 Activity Diagram	17-18
	2.2.6 Flow Chart	18-19
	2.2.7 Sequence Diagram	20-21
	2.2.8 Component Diagram	21-22
	2.3 Tools and Technologies Used	22-23
3.	**CHAPTER 3:RESULTS AND OUTPUT**	24

	3.1 Screenshots / Outputs	24
	3.1.1 Github Repository Structure of Energy Application	24
	3.1.2 AWS EC2 Security Group Inbound Rules Configuration	25
	3.1.3 Docker Image Pull and Container Deployment on AWS EC2	25-26
	3.1.4 Running Docker Container on AWS EC2 Instance	26
	3.1.5 Docker Run Command for Container Deployment	26
	3.1.6 Docker Image Push to Docker Hub	27
	3.1.7 User Interface of Energy Application	27
	3.1.8 Generated Energy Intelligence Report Output	28
	3.1.9 Explore Further Section of Energy Application	28-29
	3.2 Reports / Dashboards / Models	29-30
	3.3 Key Outcomes	30
4.	**CHAPTER 4:CONCLUSION**	31-32
5.	**CHAPTER 5:FUTURE SCOPE AND ENHANCEMENTS**	33-34

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 2.2.2	DFD	12-14
Figure 2.2.3	Use Case	14-15
Figure 2.2.4	Class Diagram	15-16
Figure 2.2.5	Activity Diagram	17-18
Figure 2.2.6	Flow Chart	18-19
Figure 2.2.7	Sequence Diagram	20-21
Figure 2.2.8	Component Diagram	21-22
Figure 3.1.1	Github Repository Structure of Energy Application	24
Figure 3.1.2	AWS EC2 Security Group Inbound Rules Configuration	25
Figure 3.1.3	Docker Image Pull and Container Deployment on AWS EC2	25-26
Figure 3.1.4	Running Docker Container on AWS EC2 Instance	26
Figure 3.1.5	Docker Run Command for Container Deployment	26
Figure 3.1.6	Docker Image Push to Docker Hub	27
Figure 3.1.7	User Interface of Energy Application	27
Figure 3.1.8	Generated Energy Intelligence Report Output	28
Figure 3.1.9	Explore Further Section of Energy Application	28-29

CHAPTER 1: PROBLEM STATEMENT & OBJECTIVES

1.1 PROJECT STATEMENT

In today's rapidly evolving software development environment, organizations are required to deliver high-quality applications at a faster pace while maintaining reliability, scalability, and consistency. Traditional software development and deployment practices involve manual processes for building, testing, and deploying applications, which often lead to inefficiencies, delays, and increased chances of human error.The Energy Application, developed using Python/ Flask API, is designed to manage and monitor energy-related data such as consumption, reporting, and system analytics. However, without an automated system in place, deploying updates and maintaining consistency across different environments—such as development, testing, and production becomes a challenging and error-prone task.One of the major challenges in the current system is the lack of Continuous Integration (CI) and Continuous Deployment (CD) practices. Developers manually integrate their code into the main repository, which can lead to integration conflicts, delayed bug detection, and unstable builds. Additionally, manual testing processes slow down the development cycle and reduce overall productivity.Another critical issue is the absence of a standardized deployment mechanism. Without containerization, the application may behave differently across environments due to variations in system configurations, dependencies, and runtime settings. This inconsistency can result in deployment failures and increased debugging efforts.Furthermore, traditional infrastructure setup requires manual provisioning of servers and resources, which is time-consuming and difficult to manage. Scaling the application to handle increased user demand becomes inefficient without automated infrastructure management. This creates a need for Infrastructure as Code (IaC) solutions that can automate resource provisioning and ensure repeatability.To address these challenges, there is a need to design and implement a robust CI/CD pipeline that automates the entire software development lifecycle—from code integration to deployment. The pipeline should integrate modern DevOps tools such as Git for version control, Jenkins or GitHub Actions for automation, Docker for containerization, AWS for orchestration, and for cloud deployment.Additionally, the use of Terraform as an Infrastructure as Code tool enables automated provisioning and management of cloud resources on AWS. This ensures that infrastructure is consistent, scalable, and easily reproducible across different environments.The proposed solution aims to eliminate manual intervention, reduce errors, improve deployment speed, and ensure consistent application performance. By implementing a CI/CD pipeline integrated with AWS and Terraform, the Energy Application can achieve continuous delivery, high availability, scalability, and improved system reliability.

To better understand the challenges faced in the existing system, the following key problems are identified:

1. **Lack of Continuous Integration (CI)**
   Developers integrate code manually, which leads to delayed detection of errors and

integration conflicts.

2. **Manual Testing Process**
   Testing is not automated, resulting in slower development cycles and increased chances of bugs reaching production.

3. **Inconsistent Deployment Environments**
   Differences in system configurations and dependencies cause the application to behave differently across environments.

4. **Absence of Containerization**
   Without Docker, the application lacks portability and cannot ensure consistent runtime environments.

5. **Manual Infrastructure Provisioning**
   Setting up servers and resources manually on AWS is time-consuming and not scalable.

1.2 PROJECT OBJECTIVES

The primary objective of this project is to design and implement a fully automated CI/CD pipeline for the Energy Application using modern DevOps practices. The system aims to streamline the software development lifecycle by automating code integration, testing, containerization, and deployment. By integrating cloud services like AWS and infrastructure automation tools like Terraform, the project ensures scalability, reliability, and consistency across environments.

The objectives of this project are outlined as follows:

1. **To implement Continuous Integration (CI)**
   Automatically integrate code changes into a shared repository and detect errors at an early stage.

2. **To implement Continuous Deployment (CD)**
   Automate the deployment process to ensure faster and reliable delivery of the application.

3. **To automate build and testing processes**
   Ensure that every code change is automatically built and tested without manual intervention.

4. **To containerize the application using Docker**
   Package the application into containers to maintain consistency across environments

5. **To deploy the application on AWS cloud**
   Use AWS services such as EC2 or EKS for scalable and reliable hosting.

6. **To use Terraform for Infrastructure as Code (IaC)**
   Automate the provisioning and management of cloud infrastructure.

7. **To ensure high scalability and availability**
   Use AWS to manage container orchestration and scaling.

8. **To reduce manual errors and improve efficiency**
   Minimize human intervention through automation.

9. **To enable faster delivery and continuous updates**
   Improve development speed and reduce time-to-market.

10. **To maintain consistency across environments**
    Ensure that the application behaves the same in development, testing, and production.

1.3 SCOPE OF THE PROJECT

The scope of this project defines the boundaries and functionalities included in the CI/CD pipeline implementation for the Energy Application. The project focuses on automating the software development lifecycle, from code integration to deployment, while ensuring scalability and reliability using cloud infrastructure and containerization.

The scope of the project includes the following:

1. **Development of CI/CD Pipeline**
   Designing and implementing an automated pipeline using Jenkins or GitHub Actions.

2. **Version Control Integration**
   Using Git and GitHub to manage source code and track changes.

3. **Automated Build and Testing**
   Building the application and executing automated test cases.

4. **Containerization using Docker**
   Creating Docker images for consistent application deployment.

5. **Cloud Deployment on AWS**
   Deploying the application on AWS EC2 or Kubernetes cluster.

6. **Infrastructure Provisioning using Terraform**
   Automating the creation and management of cloud resources.

7. **AWS Orchestration**
   Managing container deployment, scaling, and availability.

8. **Monitoring and Logging**
   Tracking system performance and debugging issues.

9. **Basic Security Implementation**
   Using authentication and secure credentials for access control.

10. **Documentation and Reporting**
    Providing detailed documentation of system design and implementation.

Out of Scope

1. Advanced AI-based monitoring systems

2. Enterprise-level distributed microservices architecture

3. Complex security frameworks like Zero Trust Architecture

CHAPTER 2:PROPOSED SOLUTION

2.1 KEY FEATURES

The system provides several important features that enhance automation, efficiency, and reliability of the application deployment process. These features are designed to reduce manual effort and ensure consistency across different environments.

The key features of the system are as follows:

1. **Automated CI/CD Pipeline**
   The system automatically performs build, testing, and deployment whenever code is pushed to the repository.

2. **Version Control Integration**
   Git and GitHub are used to manage source code, track changes, and enable collaboration among developers.

3. **Automated Testing**
   The pipeline executes automated test cases to ensure code quality and prevent faulty deployments.

4. **Docker-based Containerization**
   The application is packaged into Docker containers to ensure consistent execution across environments.

5. **Cloud Deployment on AWS**
   The application is deployed on AWS infrastructure, ensuring scalability and high availability.

6. **Infrastructure as Code using Terraform**
   Terraform is used to automatically provision and manage AWS resources, reducing manual configuration.

7. **AWS Orchestration**
   AWS manages container deployment, scaling, and load balancing.

8. **Scalability and High Availability**
   The system can automatically scale based on workload and ensures minimal downtime.

9. **Monitoring and Logging**
   The system tracks pipeline execution and application performance for debugging and analysis.

10. **Reduced Human Intervention**
    Automation minimizes manual errors and improves efficiency.

2.2 OVERALL ARCHITECTURE / WORKFLOW

The overall architecture of the system follows a DevOps-based approach where all stages of the software development lifecycle are automated. The workflow integrates multiple tools to ensure smooth and continuous delivery of the application.In this system, the developer writes code and pushes it to the GitHub repository. This action triggers the CI/CD pipeline configured using Jenkins or GitHub Actions. The pipeline automatically pulls the latest code and performs the build process, including dependency installation and compilation.Once the build is successful, automated tests are executed to validate the application. If the tests pass, a Docker image is created using a Dockerfile and pushed to a Docker registry such as Docker Hub.Terraform is then used to provision infrastructure on AWS, including EC2 instances. The AWS cluster pulls the Docker image and deploys it as containers. Finally, the application is exposed to users through a service or load balancer.

2.1.1 SYSTEM ARCHITECTURE

The main layers involved in the system are as follows:

1. Development Layer

This is the initial layer where the application is developed using Python and Flask.

- **A.** Developers write backend code using Python (Flask API)
- **B.** Frontend is developed using HTML, CSS, and JavaScript
- **C.** Code is tested locally before pushing
- **D.** Application logic and APIs are implemented

2. Version Control Layer

This layer manages source code and tracks all changes.

- **A.** Git is used for version control
- **B.** GitHub stores the project repository
- **C.** Developers push code changes to GitHub
- **D.** Webhooks trigger CI/CD pipeline automatically

3. Continuous Integration (CI) Layer

This layer automates build and testing processes.

- **A.** Jenkins or GitHub Actions is used
- **B.** Pipeline pulls latest Python code
- **C.** Dependencies are installed using pip
- **D.** Automated tests are executed
- **E.** Build status is generated

4. Containerization Layer

This layer packages the Flask application into containers.

- **A.** Docker is used to create container images
- **B.** Dockerfile defines Python environment
- **C.** Flask app and dependencies are packaged
- **D.** Ensures consistency across environments

5. Registry Layer

This layer stores Docker images.

- **A.** Docker Hub is used as registry
- **B.** Images are pushed after build
- **C.** Versioning of images is maintained
- **D.** Images can be reused or rolled back

6. Infrastructure Layer (Terraform + AWS)

This layer provisions cloud infrastructure.

- **A.** Terraform defines AWS resources
- **B.** EC2 instances or Kubernetes clusters are created
- **C.** Infrastructure setup is automated
- **D.** Ensures scalability and repeatability

7. Orchestration Layer (Kubernetes)

This layer manages containers.

- **A.** AWS deploys Flask app containers
- **B.** Pods are created and managed
- **C.** Auto-scaling is enabled
- **D.** Fault tolerance is handled

8. Deployment Layer

This layer makes the application live.

- **A.** AWS pulls image from Docker Hub
- **B.** Containers are deployed as pods
- **C.** Services expose the Flask API
- **D.** Application becomes accessible

9. Monitoring Layer

This layer ensures system performance.

- **A.** Logs are monitored
- **B.** Pipeline status is tracked
- **C.** Errors are identified
- **D.** System performance is analyzed

2.2.2 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system. It shows the flow of information between different components such as processes, data stores, and external entities. A DFD helps in understanding how input data is processed and transformed into output within a system.In this project, the DFD represents the flow of data in the CI/CD pipeline integrated with Terraform, Docker, and AWS for deploying the Energy Application.

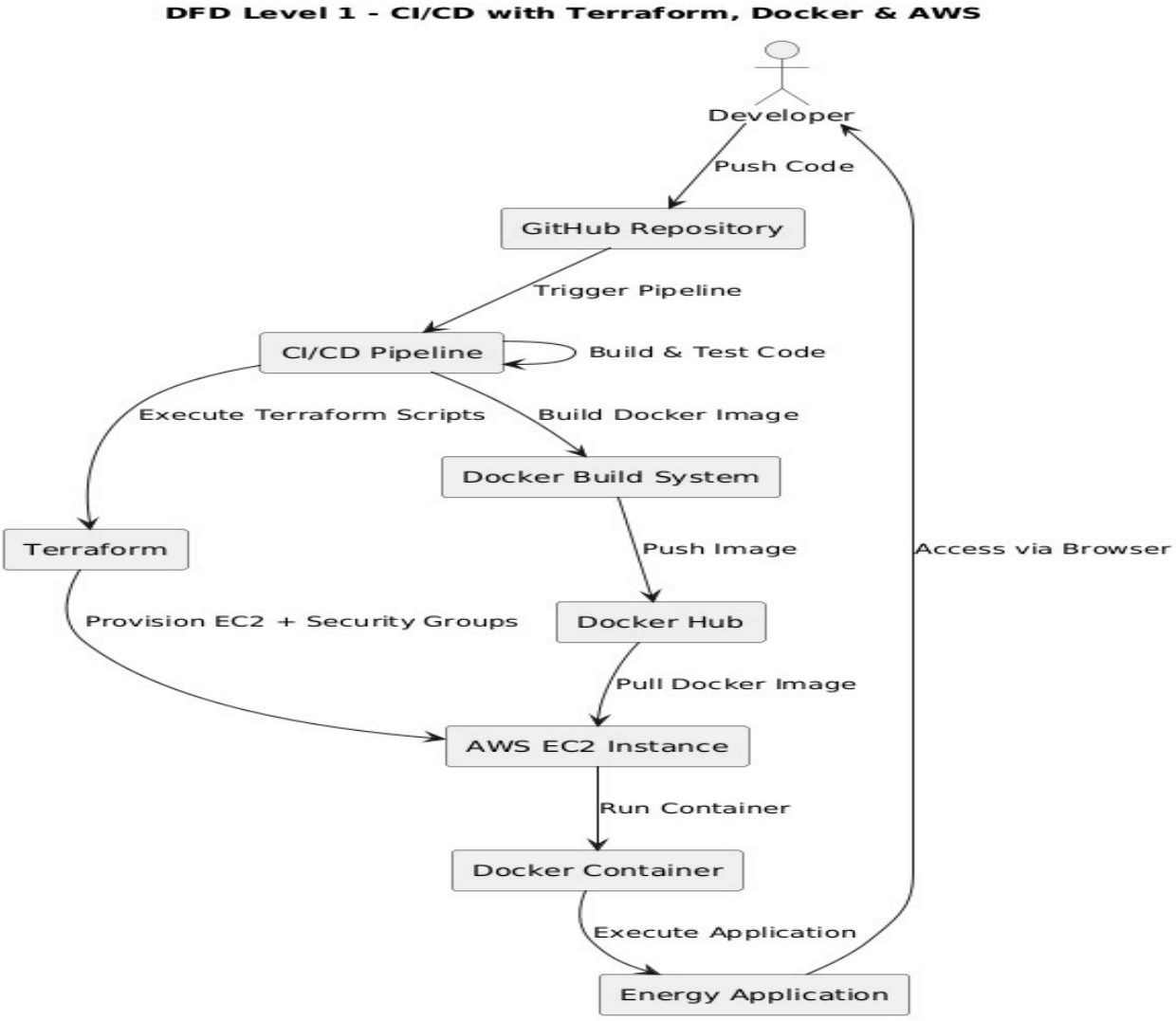


Figure 2.2.2

Working:

1. Developer-The process begins with the developer who writes the application code.

- **A.** Developer creates or updates the code
- **B.** Pushes the code to GitHub repository
- **C.** Initiates the CI/CD workflow

2. GitHub Repository-GitHub acts as the central storage for the application code.

- **A.** Stores the latest version of the code
- **B.** Tracks all changes and commits
- **C.** Triggers the CI/CD pipeline automatically

3. CI/CD Pipeline-The CI/CD pipeline automates the build and deployment process.

- **A.** Pulls the latest code from GitHub
- **B.** Builds the application
- **C.** Executes test cases
- **D.** Initiates Docker image creation
- **E.** Triggers Terraform scripts

4. Terraform (Infrastructure Layer)-Terraform is used to provision infrastructure on AWS.

- **A.** Executes Terraform scripts
- **B.** Creates AWS EC2 instance
- **C.** Configures security groups (ports access)
- **D.** Ensures infrastructure is ready for deployment

5. Docker Build System-Docker is used to containerize the application.

- **A.** Builds Docker image from application code
- **B.** Packages dependencies and environment
- **C.** Prepares application for deployment

6. Docker Hub (Registry)-Docker Hub stores the Docker image.

- **A.** Receives pushed Docker image
- **B.** Maintains versioned images
- **C.** Makes image available for deployment

7. AWS EC2 Instance-AWS EC2 acts as the hosting environment.

- **A.** Pulls Docker image from Docker Hub
- **B.** Provides cloud infrastructure
- **C.** Prepares system for running container

8. Docker Container-The application runs inside a Docker container.

- **A.** Container is created from Docker image
- **B.** Ports are mapped (e.g., 8000, 7860)
- **C.** Application starts running

9. Energy Application (Final Output)-This is the final deployed application.

- **A.** Application is executed successfully
- **B.** Users can access it via browser
- **C.** Displays energy insights and reports

10. User Access-The final step allows access to the application.

- **A.** User accesses application via public IP
- **B.** Data is processed and results are displayed
- **C.** Continuous feedback loop to developer

2.2.3 USE CASE DIAGRAM

A Use Case Diagram is a type of UML (Unified Modeling Language) diagram that represents the interaction between users (actors) and the system. It shows what actions users can perform and how they interact with the system functionalities.In this project, the Use Case Diagram represents how the Developer and Admin interact with the CI/CD system to build, test, deploy, and monitor the Python Flask Energy Application on AWS.

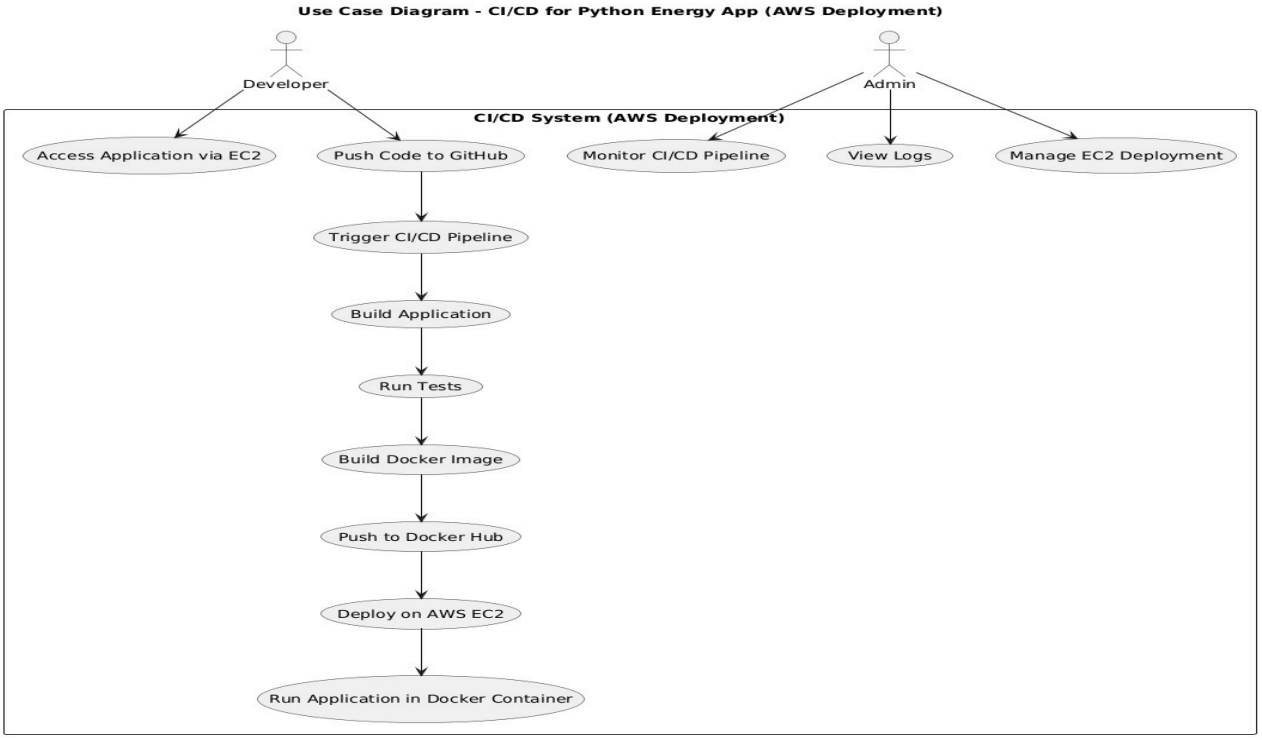


Figure 2.2.3

Working:

1. Actors in the System-Actors are the users who interact with the system.

A. **Developer-**The developer is responsible for writing code and triggering the CI/CD pipeline.Pushes code to GitHub.Triggers the CI/CD pipeline.Accesses the deployed application via EC2

B. **Admin-**The admin manages and monitors the system.Monitors CI/CD pipeline.Views logs for debugging.Manages AWS EC2 deployment

2. Use Cases (System Functionalities)-The use cases represent the actions performed within the system.

A. **Developer Use Cases-**Push Code to GitHub.Access Application via EC2

B. **CI/CD System Use Cases-**These represent automated steps performed by the system:Trigger CI/CD Pipeline,Build Application,Run Tests,Build Docker Image,Push to Docker Hub,Deploy on AWS EC2,Run Application in Docker Container

C. **Admin Use Cases-**Monitor CI/CD Pipeline,View Logs,Manage EC2 Deployment:

2.2.4 CLASS DIAGRAM

A Class Diagram is a type of UML diagram that represents the structure of a system by showing its classes, attributes, and relationships between them. It helps in understanding how different components of the system are organized and how they interact with each other.In this project, the class diagram represents the structure of the CI/CD pipeline system for the Python Flask Energy Application, including entities like repository, pipeline jobs, Docker images, containers, and AWS instances.

Working:

1. Classes and Their Attributes-Each class represents a major component in the system along with its attributes.

A. **Repository Class-**This class represents the GitHub repository where the source code is stored.Repo_ID,Repo_Name,Branch_Name,Owner,Last_Commit_ID

B. **Pipeline_Job Class-**This class represents each execution of the CI/CD pipeline.Job_ID,Commit_ID,Build_Status,Trigger_Time,Completion_Time,Triggered_By

C. **Docker_Image Class-**This class represents the Docker image created during the

pipeline.Image_ID,Image_Name,Version_Tag,Build_Time,Registry_URL

2. Relationships Between Classes-The relationships define how different components interact with each other.

A. **Repository → Pipeline_Job (1:N)-**One repository can trigger multiple pipeline jobs

B. **Pipeline_Job → Docker_Image (1:1)-**Each pipeline job creates one Docker image

C. **Docker_Image → Docker_Container (1:N)-**One image can run multiple containers

D. **Docker_Container → EC2_Instance (N:1)-**Multiple containers run on one EC2 instance

E. **Docker_Container → Application-**Container executes the application

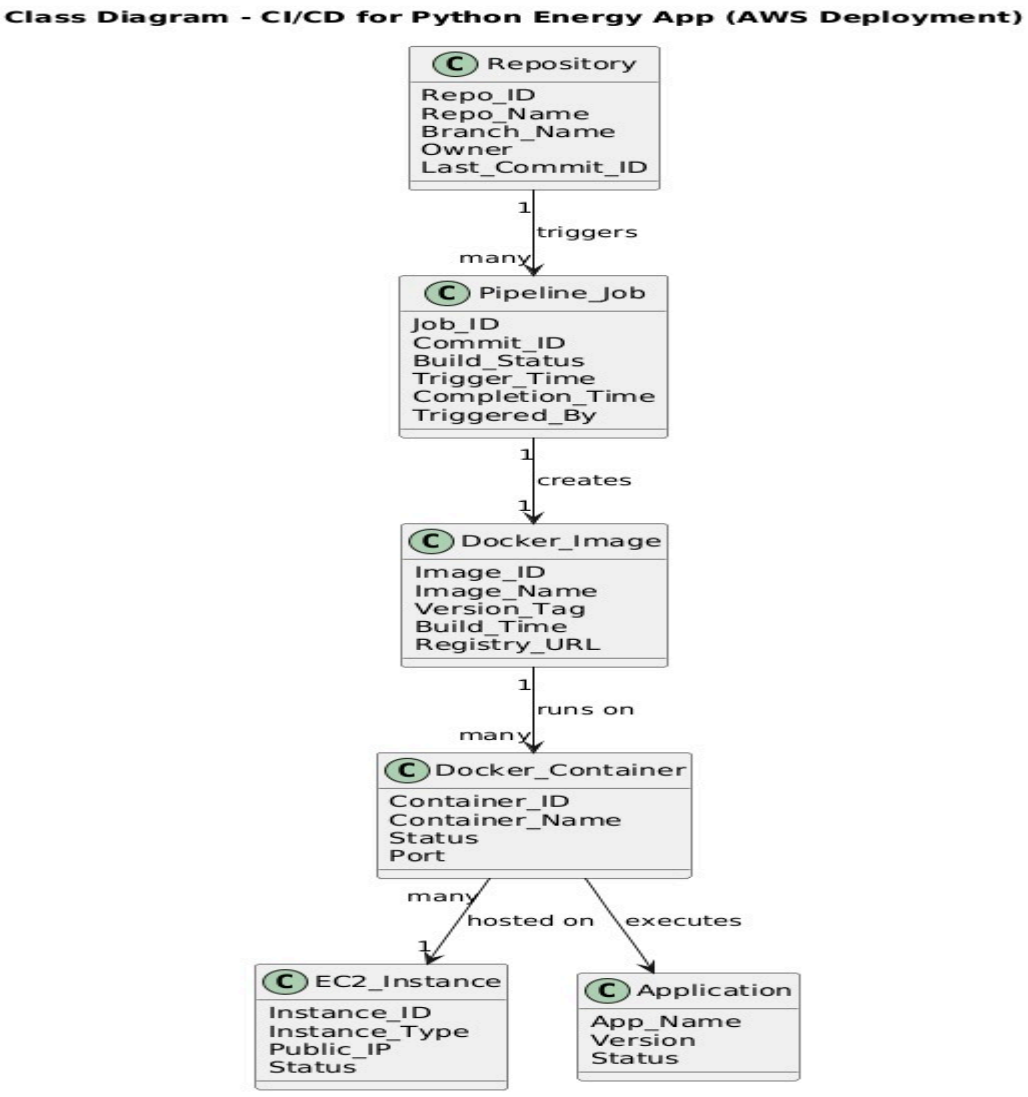


Figure 2.2.4

2.2.5 ACTIVITY DIAGRAM

An Activity Diagram is a UML diagram that represents the flow of activities or steps in a process. It shows how a system performs a sequence of actions from start to end, including decision points and parallel flows.In this project, the activity diagram illustrates the complete workflow of the CI/CD pipeline for deploying the Python Flask Energy Application on AWS EC2.

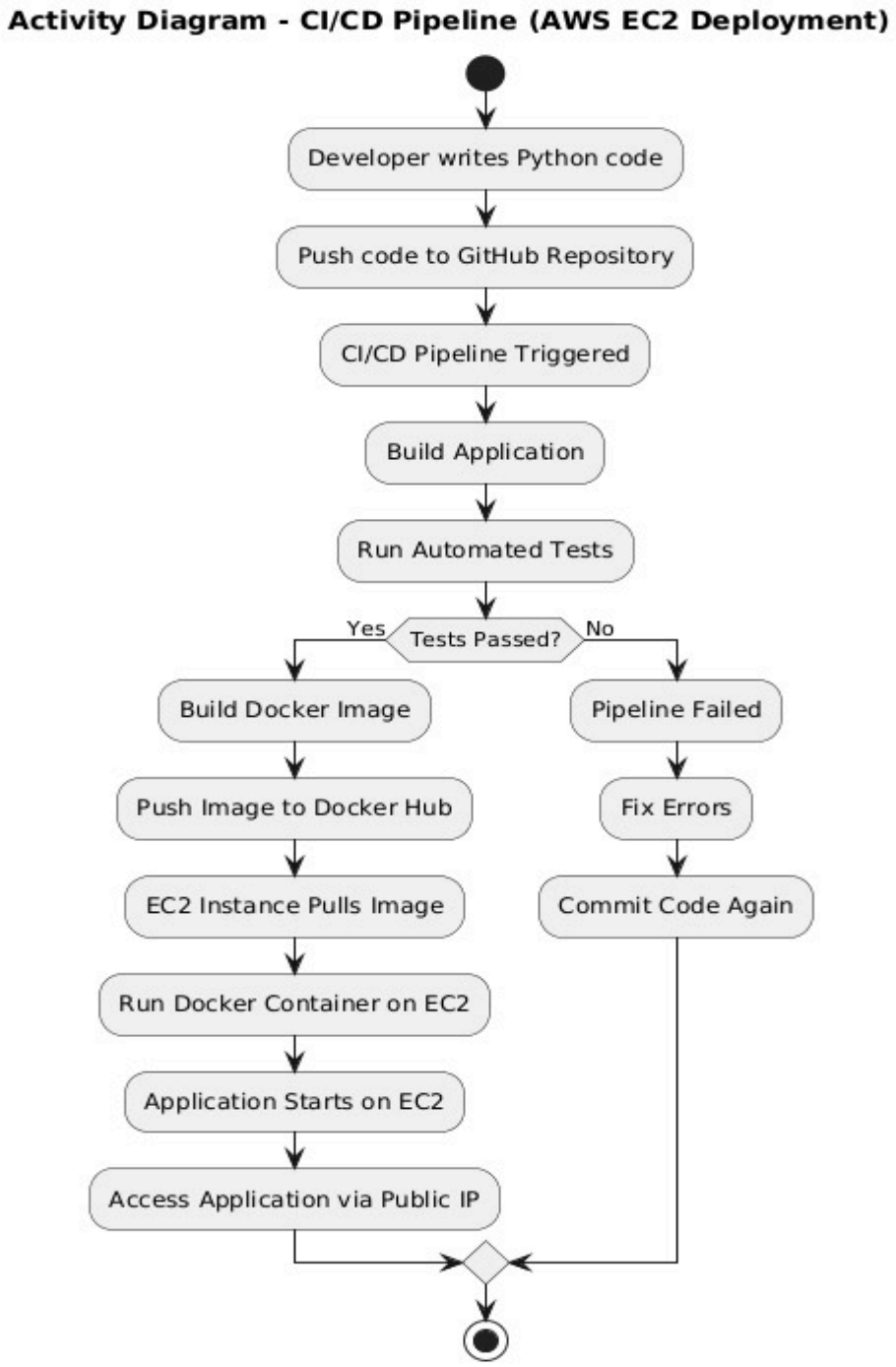


Figure 2.2.5

Working:

1.Start of Process-The process begins with the developer writing the application code.Developer writes Python (Flask) code.Prepares application for deployment

2.Push Code to Repository-The developer pushes code to GitHub.Code is uploaded to repository.Version control tracks changes.Pipeline trigger is initiated

3.CI/CD Pipeline Triggered-The pipeline starts automatically after code push.CI/CD tool detects new commit.Pipeline execution begins.Code is pulled from repository

4.Build Application-The application is built.Dependencies are installed (requirements.txt).Application environment is prepared.Build process completes

5.Run Automated Tests-Testing ensures code quality.Test cases are executed.Code correctness is verified.System checks for errors

6.Decision: Tests Passed?-This is the most important decision point.

2.2.6 FLOW CHART

A Flowchart is a graphical representation of a process that shows the sequence of steps using symbols like rectangles (process), diamonds (decision), and arrows (flow). It helps in understanding the logical flow of a system from start to end.In this project, the flowchart represents the complete CI/CD pipeline workflow integrated with Docker, Terraform, and AWS for deploying the Python Flask Energy Application.

Working:

1. Start of Process-The process begins with the developer writing the application code.Developer writes Python (Flask) code.Prepares application for deployment

2. Push Code to GitHub-The developer uploads the code to the repository.Code is pushed to GitHub.Version control tracks changesCI/CD pipeline is triggered automatically

3.CI/CD Pipeline Triggered-The pipeline starts execution.Code is pulled from repository.Pipeline begins build process

4.Build Application-The application is prepared for execution.Dependencies are installedApplication is compiled and configured

5. Run Tests-Testing ensures code quality.Automated test cases are executed.System checks

for errors.Ensures application stability

6. Decision: Tests Passed?-This is a decision point in the flowchart.

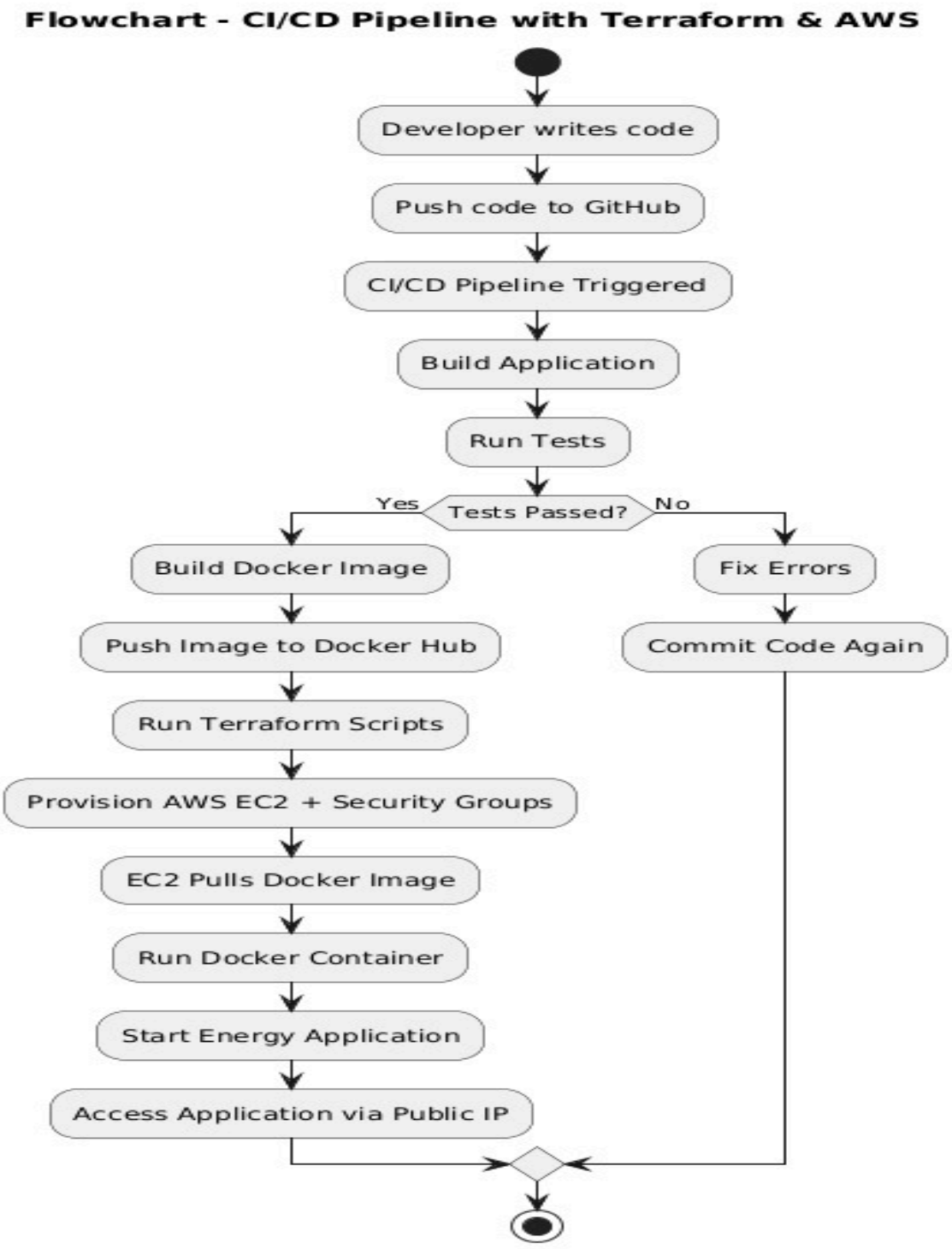


Figure 2.2.6

2.2.7 SEQUENCE DIAGRAM

A Sequence Diagram is a UML diagram that shows how different components of a system interact with each other in a specific sequence over time. It represents the order of messages exchanged between system elements to complete a process.In this project, the sequence diagram illustrates the interaction between Developer, GitHub, CI/CD pipeline, Docker, AWS EC2, and the Energy Application during the deployment process.

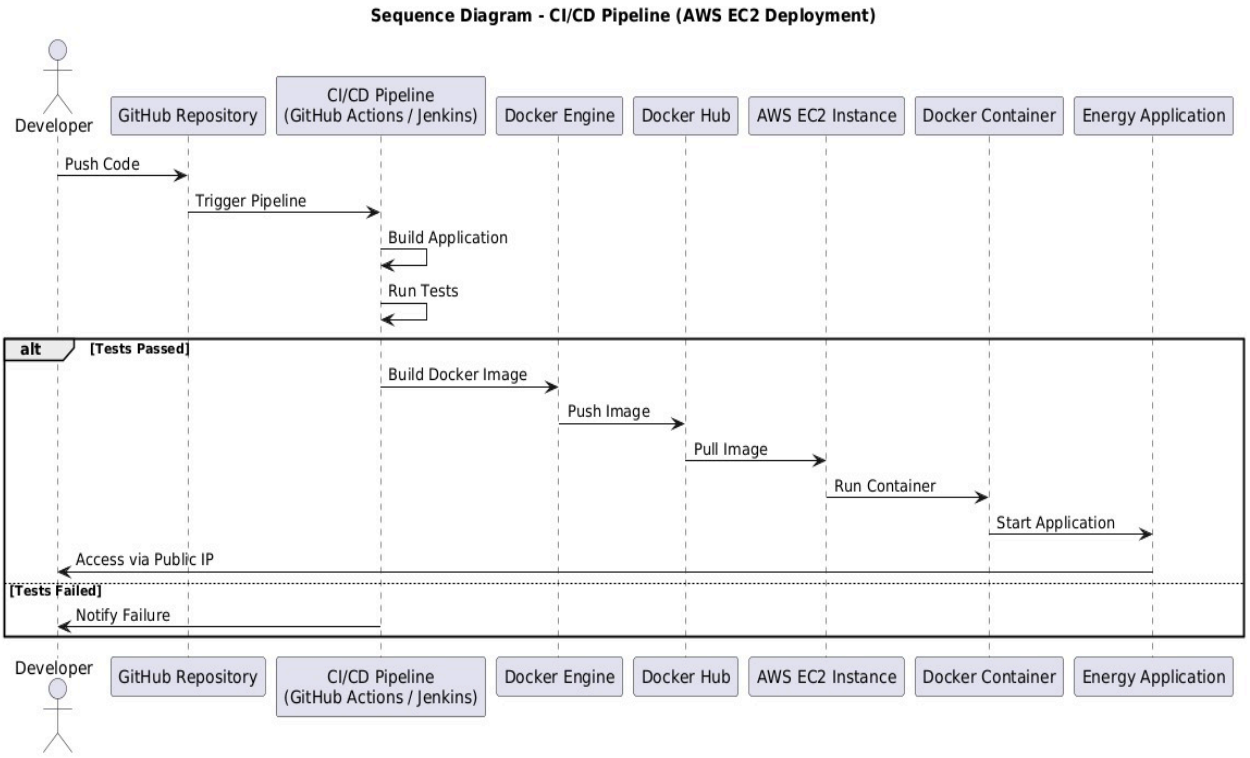


Figure 2.2.7

Working:

1. Components (Participants)-The main components involved in the sequence are:

A. **Developer –** Writes and pushes code
B. **GitHub Repository –** Stores code and triggers pipeline
C. **CI/CD Pipeline (Jenkins/GitHub Actions) –** Automates build and deployment
D. **Docker Engine –** Builds Docker image
E. **Docker Hub –** Stores Docker image
F. **AWS EC2 Instance –** Hosts application
G. **Docker Container –** Runs application
H. **Energy Application –** Final output

2. Step-by-Step Interaction-The sequence of interactions is as follows:

A. **Code Push-**Developer pushes code to GitHub.GitHub receives and stores updated code
B. **Pipeline Trigger-**GitHub triggers CI/CD pipeline.Pipeline starts execution automatically

3. Build and Test-CI/CD pipeline builds the application.Runs automated tests.Verifies code quality

4. Decision: Tests Passed?

2.2.8 COMPONENT DIAGRAM

A Component Diagram is a UML diagram that shows the physical structure of a system by representing its components and how they interact with each other. It focuses on high-level system architecture and how different modules or services are connected.In this project, the component diagram represents the architecture of the CI/CD pipeline for the Python Flask Energy Application, including development, version control, CI/CD system, containerization, infrastructure provisioning, and AWS deployment.

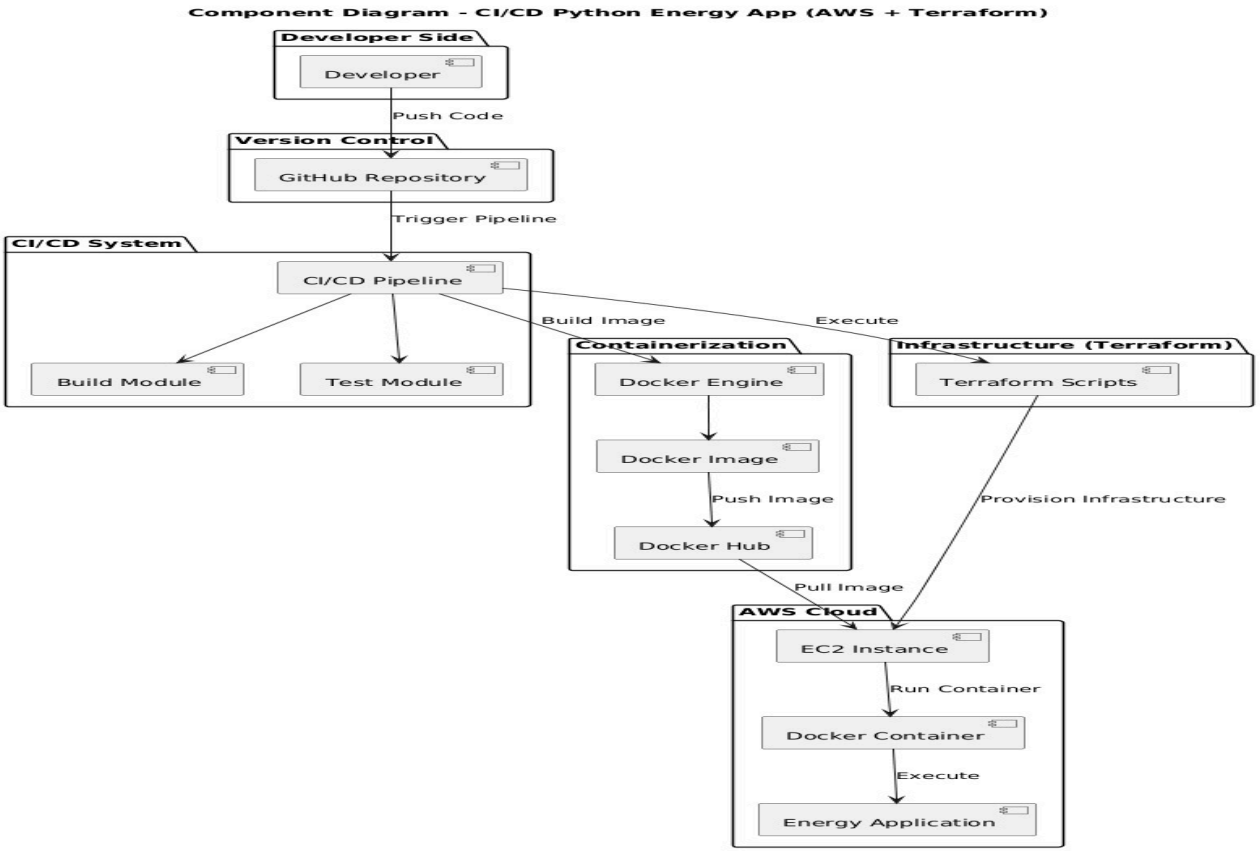


Figure 2.2.8

Working:

1. Developer Side-This is the starting point of the system where development takes place.Developer writes Python Flask code.Prepares application for deployment.Pushes code to version control system

2. Version Control (GitHub Repository)-This component manages source code and triggers the pipeline.Stores application code.Maintains version history.Triggers CI/CD pipeline on code push

3. CI/CD System-This is the core component that automates the workflow.

4. Containerization (Docker)-This component handles packaging of the application.Docker Engine builds container image.Docker Image stores application and dependencies.Image is pushed to Docker Hub

5. Infrastructure (Terraform)-This component manages cloud infrastructure..Terraform scripts define AWS resources.Provisions EC2 instance and security groups.Automates infrastructure setup

6. AWS Cloud-This is the deployment environment where the application runs.EC2 instance is created.Docker image is pulled from Docker Hub.Container is executed

7. Docker Container-This component runs the application.Container is created from Docker image.Ports are mapped for access.Application is executed inside container

8. Energy Application-This is the final output of the system.Application starts successfully.Processes user requests.Provides energy insights and reports

2.3 TOOLS AND TECHNOLOGIES USED

1. **Frontend**

A. **HTML-**Used to create the structure of web pages.
B. **CSS-**Used to style and design the user interface.
C. **JavaScript-**Adds interactivity and dynamic behavior to the web application.

2. **Backend**

A. **Python-**A high-level programming language used to develop the backend logic of the application.
B. **Flask API-**A lightweight web framework in Python used to build REST APIs for handling requests and responses.

3. Version Control

A. **Git-**A version control system used to track changes in source code.
B. **GitHub-**A platform to store code repositories and collaborate with team members.

4. CI/CD Tools

A. **Jenkins-**An automation tool used to build, test, and deploy applications.
B. **GitHub Actions-**A CI/CD tool that automates workflows directly from GitHub.

5. Containerization

A. **Docker-**Used to package the application into containers for consistent deployment.
B. **Dockerfile-**A script containing instructions to build a Docker image.
C. **Docker Hub-**A cloud repository to store and manage Docker images.

6. Cloud Platform

A. **AWS (Amazon Web Services)-**Provides cloud infrastructure for hosting and deploying applications.
B. **AWS EC2-**Virtual servers used to run applications.
C. **AWS EKS-**A managed AWS service for container orchestration.

7. Infrastructure as Code

A. **Terraform-**Used to automate the creation and management of cloud infrastructure.

8. Orchestration

A. **AWS-**Manages deployment, scaling, and operation of containerized applications.

CHAPTER 3:RESULTS AND OUTPUT

The Results and Output section presents the final outcomes of the implemented CI/CD pipeline for the Python Flask Energy Application. This section demonstrates how the system performs after integrating all DevOps tools and deploying the application on AWS using Terraform. It highlights the successful execution of the pipeline, deployment of the application, and the overall improvements achieved through automation.

3.1 SCREENSHOTS / OUTPUTS

3.1.1 GitHub Repository Structure of Energy Application

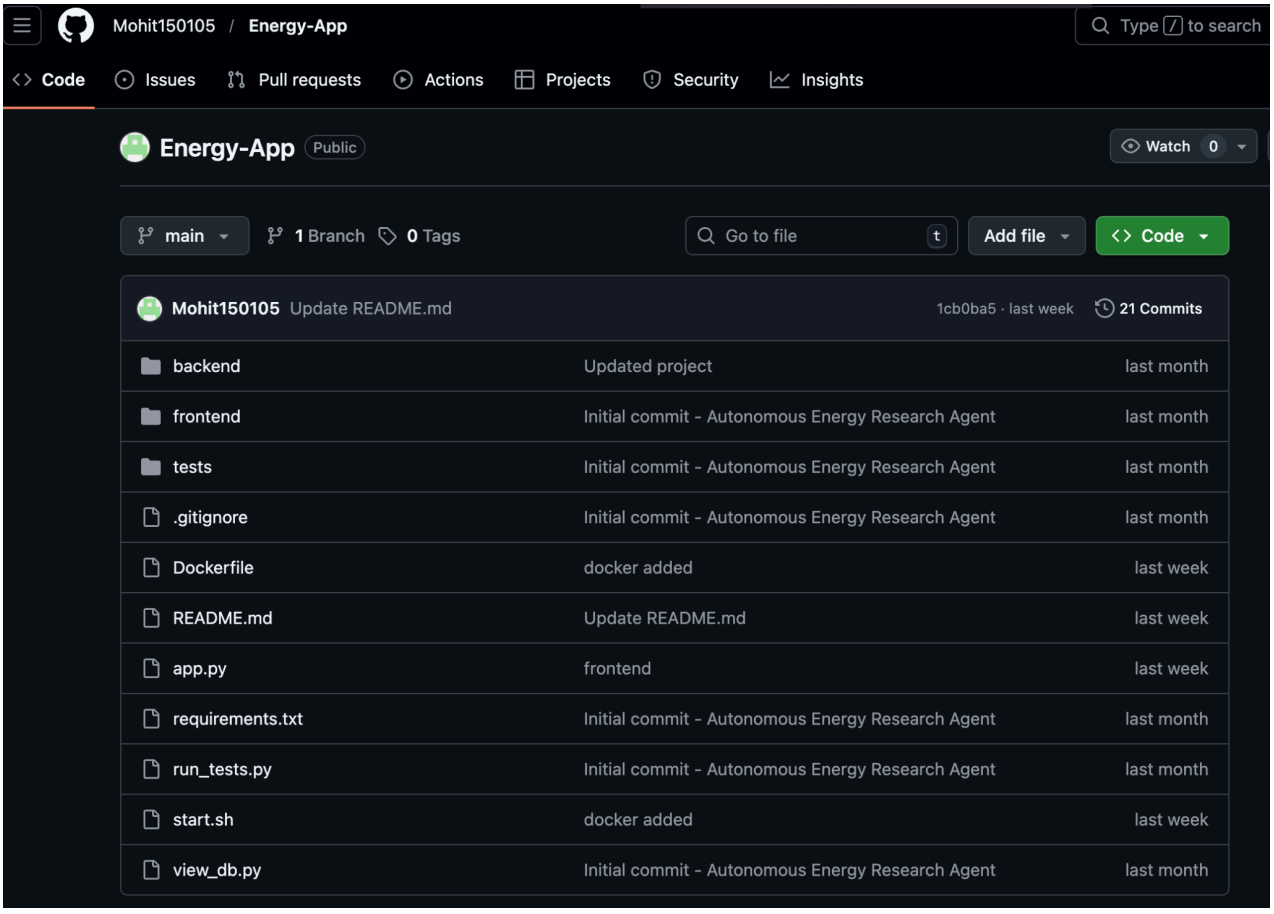


Figure 3.1.1

This figure shows the GitHub repository of the Energy Application project. It contains the complete project structure including backend, frontend, and testing modules. Important files such as app.py, requirements.txt, Dockerfile, and shell scripts are included for application development, dependency management, and containerization.The repository maintains version control with multiple commits and serves as the central source for triggering the CI/CD pipeline. It ensures proper organization of code and enables collaboration among developers.

3.1.2 AWS EC2 Security Group Inbound Rules Configuration

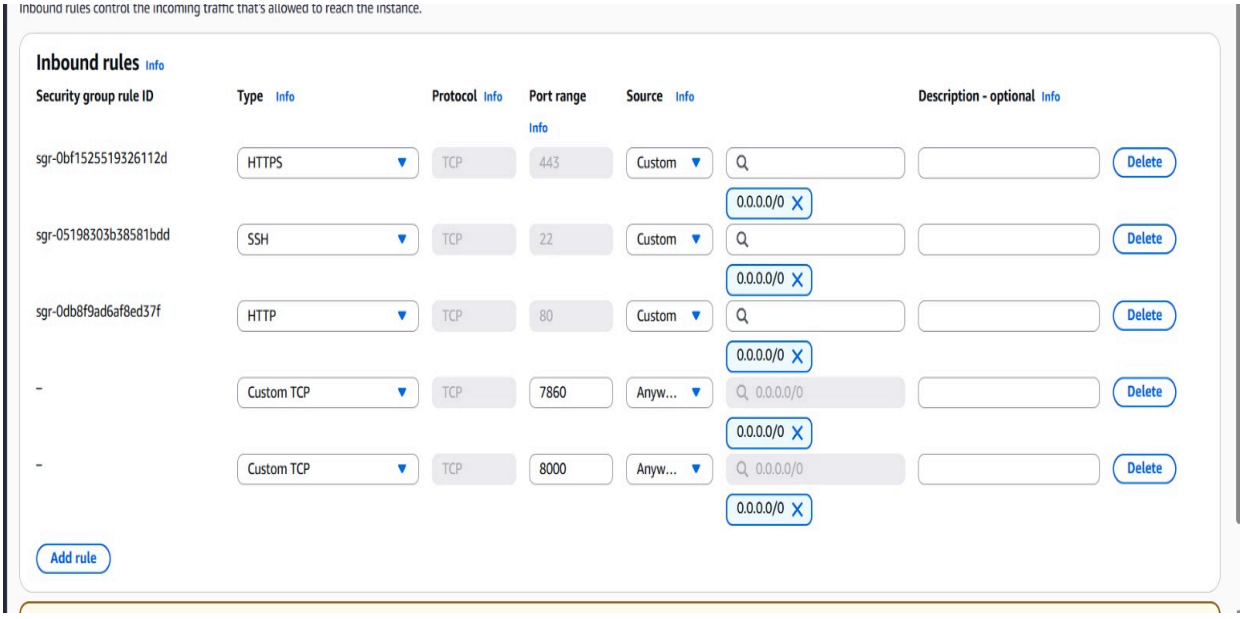
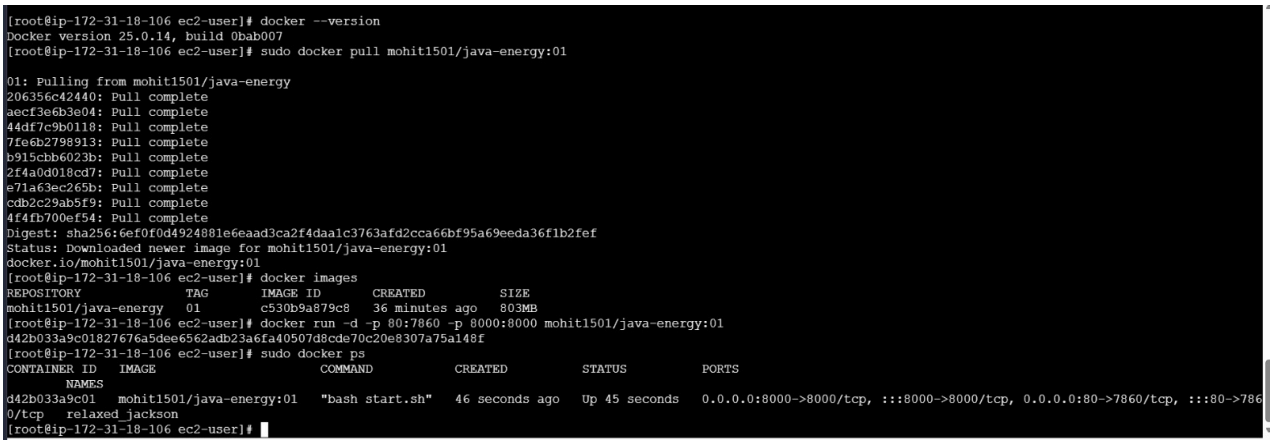


Figure 3.1.2

This figure shows the configuration of inbound rules for the AWS EC2 instance used in deploying the Energy Application. The security group allows incoming traffic on essential ports such as HTTP (80), HTTPS (443), and SSH (22), along with custom application ports (8000 and 7860).All ports are configured with source 0.0.0.0/0, enabling public access to the application. This setup ensures that users can access the deployed application through a web browser and allows secure remote access for management and deployment purposes.

3.1.3 Docker Image Pull and Container Deployment on AWS EC2



```
[root@ip-172-31-18-106 ec2-user]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-18-106 ec2-user]# sudo docker pull mohit1501/java-energy:01

01: Pulling from mohit1501/java-energy
206356c42440: Pull complete
aecf3e6b3e04: Pull complete
44df7c9b0118: Pull complete
7fe6b2798913: Pull complete
b915cbb6023b: Pull complete
2f4a0d018cd7: Pull complete
e71a63ec265b: Pull complete
cdb2c29ab5f9: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:6ef0f0d4924881e6eaad3ca2f4daa1c3763afd2cca66bf95a69eeda36f1b2fef
Status: Downloaded newer image for mohit1501/java-energy:01
docker.io/mohit1501/java-energy:01
[root@ip-172-31-18-106 ec2-user]# docker images
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
mohit1501/java-energy   01        c530b9a879c8   36 minutes ago   803MB
[root@ip-172-31-18-106 ec2-user]# docker run -d -p 80:7860 -p 8000:8000 mohit1501/java-energy:01
d42b033a9c01827676a5dee6562adb23a6fa40507d8cde70c20e8307a75a148f
[root@ip-172-31-18-106 ec2-user]# sudo docker ps
CONTAINER ID   IMAGE                     COMMAND           CREATED          STATUS          PORTS
        NAMES
d42b033a9c01   mohit1501/java-energy:01   "bash start.sh"   46 seconds ago   Up 45 seconds   0.0.0.0:8000->8000/tcp, :::8000->8000/tcp, 0.0.0.0:80->7860/tcp, :::80->786
0/tcp   relaxed_jackson
[root@ip-172-31-18-106 ec2-user]#
```

Figure 3.1.3

This figure shows the process of pulling a Docker image from Docker Hub and deploying it on an AWS EC2 instance. The terminal output displays the successful execution of the docker pull command, followed by listing of Docker images and running the container using the docker run

command.It also shows the docker ps output confirming that the container is running successfully with proper port mappings. This demonstrates that the application image has been successfully retrieved, deployed, and executed on the cloud environment.

3.1.4 Running Docker Container on AWS EC2 Instance

```
[ec2-user@ip-172-31-18-137 ~]$ sudo docker ps
CONTAINER ID   IMAGE                    COMMAND          CREATED        STATUS         PORTS
       NAMES
e95b0e526af7   mohit1501/java-energy:01   "bash start.sh"   48 seconds ago   Up 47 seconds   0.0.0.0:8000->8000/tcp, :::8000->8000/tcp, 0.0.0.0:80->7860/tcp, :::80->786
0/tcp   energy-agent
[ec2-user@ip-172-31-18-137 ~]$
```

Figure 3.1.4

This figure shows the output of the docker ps command executed on the AWS EC2 instance, displaying the running Docker container for the Energy Application. It includes details such as container ID, image name, command, creation time, status, and port mappings.The container is running successfully, with ports mapped (8000 and 7860) to allow external access. This confirms that the application has been successfully deployed and is actively running inside a Docker container on the cloud server.

3.1.5 Docker Run Command for Container Deployment

```
[ec2-user@ip-172-31-18-137 ~]$ docker run -d \
-p 80:7860 \
-p 8000:8000 \
--restart unless-stopped \
--name energy-agent \
mohit1501/java-energy:01
e95b0e526af790d973d798931a97a02918f1b8bd826c7da56f5d271eb80ebce3
```

Figure 3.1.5

This figure shows the execution of the docker run command used to deploy the Energy Application container on the AWS EC2 instance. The command runs the container in detached mode with port mappings (80 → 7860 and 8000 → 8000), assigns a container name, and enables automatic restart using the --restart unless-stopped option.The successful execution confirms that the Docker container has been created and is ready to run the application.

3.1.6 Docker Image Push to Docker Hub

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

The push refers to repository [docker.io/mohit1501/java-energy]
4d29364641d8: Already exists
4f4fb700ef54: Layer already exists
2f4a0d018cd7: Layer already exists
cdb2c29ab5f9: Layer already exists
b915cbb6023b: Layer already exists
aecf3e6b3e04: Layer already exists
206356c42440: Layer already exists
7fe6b2798913: Layer already exists
e71a63ec265b: Pushing [==================================>       ] 163.6MB/190.6MB
44df7c9b0118: Layer already exists
```

Figure 3.1.6

This figure shows the process of pushing a Docker image to Docker Hub using the docker push command. The terminal output indicates that multiple image layers already exist, and only the necessary layers are being uploaded.The progress bar shows the image being successfully pushed to the Docker repository. This confirms that the Docker image is stored in the registry and is ready for deployment on cloud infrastructure.

3.1.7 User Interface of Energy Application (Input Page)

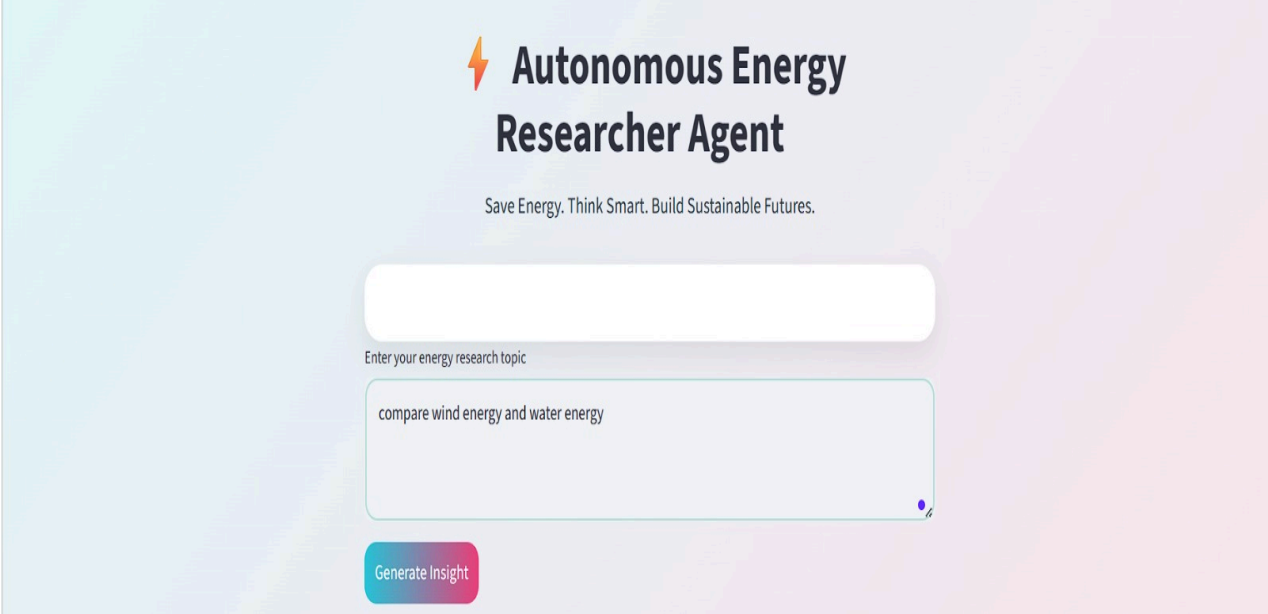
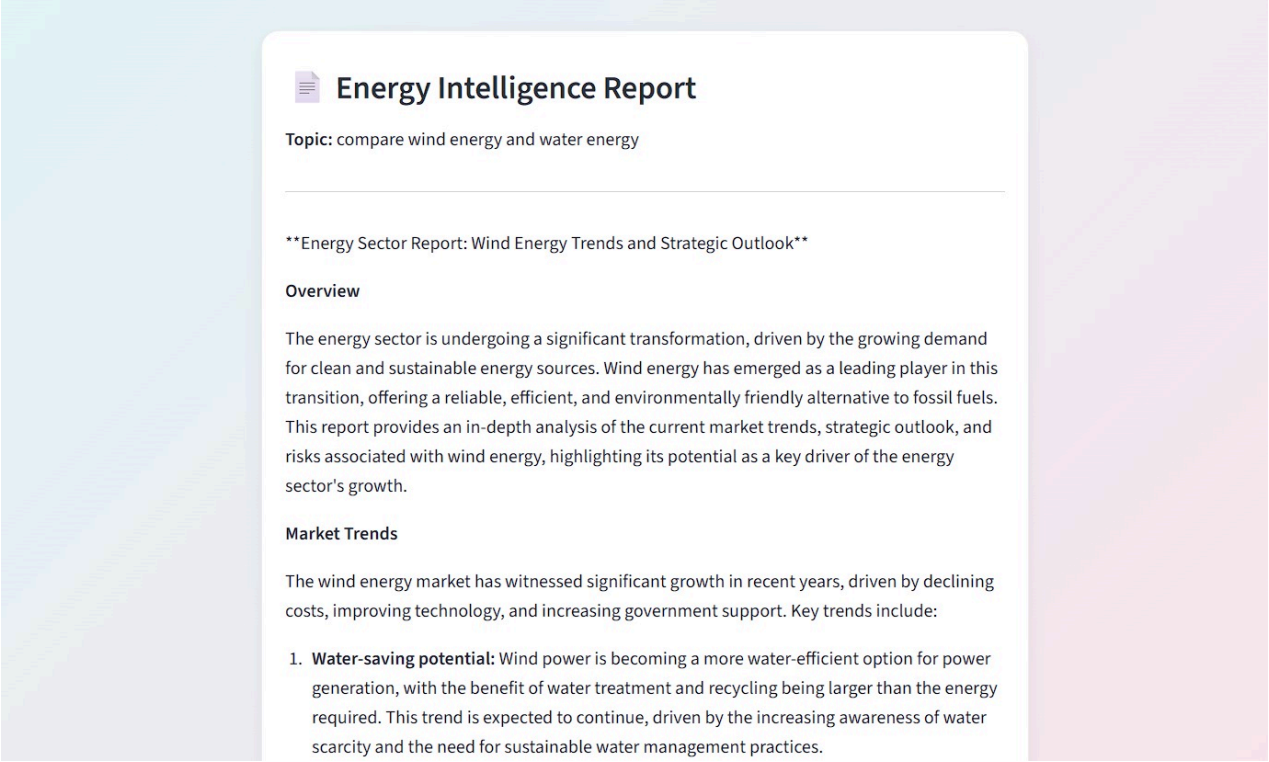


Figure 3.1.7

This figure shows the user interface of the deployed Energy Application. It allows users to enter an energy-related query in the input field, such as comparing different energy sources.

The application provides a simple and interactive interface with a "Generate Insight" button to process user input.This confirms that the frontend is successfully integrated with the backend and is accessible through the deployed system.

3.1.8 Generated Energy Intelligence Report Output



Energy Intelligence Report

Topic: compare wind energy and water energy

Energy Sector Report: Wind Energy Trends and Strategic Outlook

Overview

The energy sector is undergoing a significant transformation, driven by the growing demand for clean and sustainable energy sources. Wind energy has emerged as a leading player in this transition, offering a reliable, efficient, and environmentally friendly alternative to fossil fuels. This report provides an in-depth analysis of the current market trends, strategic outlook, and risks associated with wind energy, highlighting its potential as a key driver of the energy sector's growth.

Market Trends

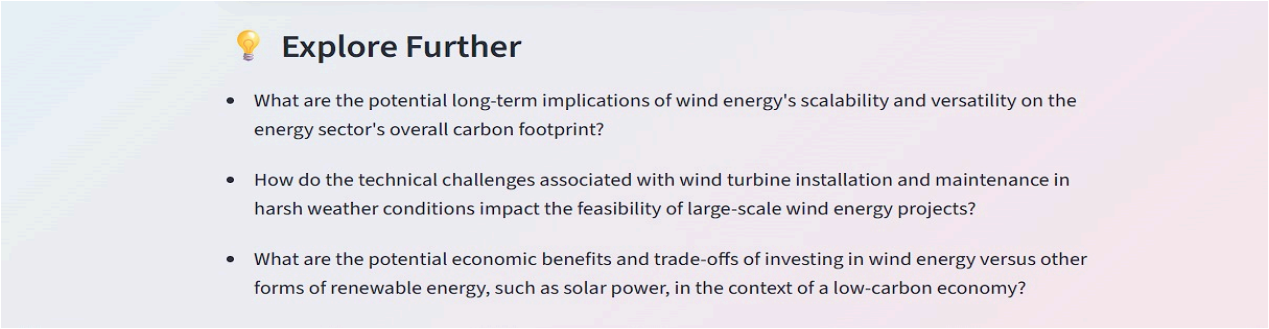
The wind energy market has witnessed significant growth in recent years, driven by declining costs, improving technology, and increasing government support. Key trends include:

1. **Water-saving potential:** Wind power is becoming a more water-efficient option for power generation, with the benefit of water treatment and recycling being larger than the energy required. This trend is expected to continue, driven by the increasing awareness of water scarcity and the need for sustainable water management practices.

Figure 3.1.8

This figure shows the output generated by the Energy Application after processing the user's input. The system produces a detailed Energy Intelligence Report that includes sections such as overview, market trends, and analytical insights related to the selected topic.The report demonstrates the successful functioning of the backend logic and confirms that the application can analyze input queries and generate meaningful, structured results for users.

3.1.9 Explore Further Section of Energy Application



Explore Further

- What are the potential long-term implications of wind energy's scalability and versatility on the energy sector's overall carbon footprint?
- How do the technical challenges associated with wind turbine installation and maintenance in harsh weather conditions impact the feasibility of large-scale wind energy projects?
- What are the potential economic benefits and trade-offs of investing in wind energy versus other forms of renewable energy, such as solar power, in the context of a low-carbon economy?

Figure 3.1.9

This figure shows the "Explore Further" section of the Energy Application, which provides additional questions and insights based on the generated report. It suggests related topics and encourages users to explore deeper aspects of energy research.This feature enhances user engagement by offering extended analysis and supports better understanding of the subject matter through guided exploration.

3.2 REPORTS / DASHBOARDS / LOGS

The CI/CD pipeline and deployed Energy Application generate various reports, logs, and outputs that help in monitoring system performance, debugging errors, and ensuring smooth execution. These reports provide visibility into each stage of the pipeline, including code integration, containerization, infrastructure provisioning, and final application output.The following reports and logs are observed in the system:

1. CI/CD Pipeline Logs

The CI/CD pipeline generates logs during build, test, and deployment stages.

- **A.** Shows execution of build and test processes
- **B.** Displays success or failure status
- **C.** Helps identify errors in pipeline
- **D.** Ensures automation is working correctly

2. Docker Build & Push Logs

Docker logs show the process of building and pushing images.

- **A.** Displays Docker image build process
- **B.** Shows layer-wise upload during push
- **C.** Confirms image stored in Docker Hub
- **D.** Helps track containerization status

3. Docker Container Logs

Docker container logs confirm successful deployment.

- **A.** Shows running containers using docker ps
- **B.** Displays container status (Up/Running)
- **C.** Shows port mappings (8000, 7860)
- **D.** Confirms application is active

4. AWS Deployment & Configuration Output

AWS outputs confirm infrastructure setup and accessibility.

- **A.** Displays EC2 instance configuration
- **B.** Shows security group rules (ports open)
- **C.** Confirms public access via IP

D. Ensures application availability

5. Application Output Reports

The application generates detailed reports based on user input.

A. Produces Energy Intelligence Report
B. Includes overview, trends, and analysis
C. Displays structured and readable output
D. Confirms backend processing

6. User Interaction Output

The system provides interactive outputs to users.

A. Input interface accepts user queries
B. "Explore Further" suggests additional insights
C. Enhances user engagement
D. Improves overall user experience

3.3 KEY OUTCOMES

1. Successful Implementation of CI/CD Pipeline

A. Automated pipeline from code push to deployment
B. Eliminates manual build and deployment steps
C. Ensures continuous integration and delivery

2. Efficient Containerization using Docker

A. Application packaged into Docker containers
B. Ensures consistency across environments
C. Simplifies deployment process

3. Successful Cloud Deployment on AWS

A. Application deployed on AWS EC2 instance
B. Public access enabled through IP and ports
C. Ensures high availability of application

4. Infrastructure Automation using Terraform

A. AWS resources provisioned automatically
B. Eliminates manual infrastructure setup
C. Ensures repeatability and scalability

CHAPTER 4:CONCLUSION

The project titled "CI/CD Pipeline for Python Flask Energy Application using AWS and Terraform" successfully demonstrates the practical implementation of modern DevOps practices to automate the software development and deployment lifecycle. In today's fast-paced technological environment, organizations require efficient, reliable, and scalable systems for delivering applications, and this project effectively addresses those requirements through automation and cloud integration.The system is designed using Python and Flask API for backend development, along with HTML, CSS, and JavaScript for the frontend interface. The integration of DevOps tools such as GitHub, Docker, AWS, and Terraform ensures that the entire workflow—from code development to deployment—is streamlined and automated. The use of a CI/CD pipeline eliminates manual intervention, thereby reducing errors and improving overall productivity.One of the major achievements of this project is the successful implementation of Continuous Integration (CI), where every code change is automatically integrated, built, and tested. This ensures that bugs and errors are detected at an early stage, improving code quality and system reliability. Similarly, Continuous Deployment (CD) enables automatic deployment of the application to the AWS environment, significantly reducing deployment time and effort.The use of Docker for containerization plays a crucial role in ensuring consistency across different environments. By packaging the application along with its dependencies into containers, the project eliminates issues related to environment mismatches. This ensures that the application behaves the same in development, testing, and production environments.Another important aspect of the project is the implementation of Infrastructure as Code (IaC) using Terraform. Terraform automates the provisioning and management of AWS resources such as EC2 instances and security groups. This not only reduces manual configuration efforts but also ensures repeatability, scalability, and efficient resource management.The deployment of the application on AWS EC2 demonstrates the use of cloud computing for hosting scalable applications. The configured security groups and port mappings allow users to access the application through a web browser, ensuring availability and usability. The successful execution of Docker containers on the cloud confirms that the system is capable of handling real-world deployment scenarios.Furthermore, the project includes proper monitoring through logs generated at different stages such as CI/CD pipeline execution, Docker operations, and AWS deployment. These logs help in identifying errors, debugging issues, and improving system performance.The Energy Application itself functions effectively by accepting user inputs and generating detailed energy intelligence reports. The inclusion of features like "Explore Further" enhances user engagement by providing additional insights and analysis. This confirms that both the frontend and backend components are well integrated and functioning as expected.

Key Highlights of the Project

1. Successful automation of the complete CI/CD pipeline
2. Efficient containerization using Docker
3. Automated infrastructure provisioning using Terraform
4. Reliable cloud deployment on AWS EC2

5. Improved system scalability and flexibility
6. Reduction in manual effort and human errors
7. Faster and more efficient deployment process
8. Proper logging and monitoring mechanisms
9. Fully functional Energy Application with user interaction
10. Real-world implementation of DevOps concepts

CHAPTER 5:FUTURE SCOPE AND ENHANCEMENTS

The current implementation of the CI/CD pipeline for the Python Flask Energy Application successfully demonstrates automation, containerization, and cloud deployment. However, with continuous advancements in technology, there are several opportunities to further enhance the system in terms of scalability, security, performance, and functionality. The future scope of this project focuses on extending the system to meet enterprise-level requirements and improving its overall capabilities.The following enhancements can be implemented in the future:

1. Kubernetes-Based Deployment and Auto-Scaling

Currently, the application is deployed on AWS EC2 using Docker containers. In the future, Kubernetes can be fully integrated to manage container orchestration more efficiently.

- **A.** Deploy application using Kubernetes clusters (EKS)
- **B.** Enable automatic scaling of pods based on traffic
- **C.** Implement load balancing for better performance
- **D.** Improve fault tolerance with self-healing mechanisms

2. Advanced Monitoring and Logging

Monitoring plays a crucial role in maintaining system performance and reliability. Advanced tools can be integrated to provide real-time insights.

- **A.** Integrate Prometheus for metrics collection
- **B.** Use Grafana for visualization dashboards
- **C.** Implement centralized logging using ELK Stack (Elasticsearch, Logstash, Kibana)
- **D.** Monitor CPU, memory, and network usage

3. CI/CD Pipeline Enhancement

The current pipeline can be further improved by adding advanced features and stages.

- **A.** Add multiple testing stages (unit, integration, security testing)
- **B.** Implement automated rollback on deployment failure
- **C.** Use parallel pipeline execution for faster builds
- **D.** Integrate code quality tools like SonarQube

4. Security Improvements

Security is a critical aspect of any application, especially in cloud environments.

- **A.** Implement HTTPS using SSL/TLS certificates
- **B.** Secure API endpoints with authentication and authorization
- **C.** Use AWS IAM roles for secure access management

D. Store sensitive data using secret management tools

5. Multi-Cloud and Hybrid Deployment

To increase reliability and reduce dependency on a single cloud provider, multi-cloud strategies can be adopted.

A. Deploy application across multiple cloud platforms (AWS, Azure, GCP)
B. Implement failover mechanisms
C. Ensure high availability and disaster recovery
D. Improve system resilience